

Блок 1: Теория вероятности и логика

Задание 1: Фермер

На ферме содержатся шесть разных видов животных, и каждый раз, когда фермер заходит в сарай, он видит одно случайное животное. За день фермер заходит в сарай 6 раз.

Каково математическое ожидание количества разных видов животных, которые фермер увидит за день?

Ответ (округлить до сотых): **3.99**

Задание 2: Кулинарное соревнование

В конкурсе участвуют 80 шеф-поваров с уникальными уровнями мастерства. В первом этапе судьи случайным образом распределяют их по парам (в любом состязании двух шефов выигрывает тот, у кого выше уровень мастерства). На втором этапе шефы снова случайно образуют пары для финального раунда (пары могут повториться). Победную награду получают те, кто выиграл в обоих этапах.

Каково математическое ожидание числа победителей?

Ответ (округлить до десятых): **26.8**

Задание 3: Одинокая дорога

На пустынном шоссе вероятность появления автомобиля за 30-минутный период составляет 0.95.

Какова вероятность его появления за 10 минут? А за 27 минут?

Ответ (дать в процентах, округлив до десятых через точку с запятой): **63.2; 93.3**

Блок 2: Python

Задание 1: Изоморфизмы

Реализовать функцию (или тело функции), которая проверяет на изоморфность два слова. Пояснение: строки s и t называются изоморфными, если все вхождения каждого символа строки s можно последовательно заменить другим символом и получить строку t . Порядок символов при этом должен сохраняться, а замена — быть уникальной. Так, два разных символа строки s нельзя заменить одним и тем же символом из строки t , а вот одинаковые символы в строке s должны заменяться одним и тем же символом.

```
# Пример:
s = 'paper'
t = 'title'
print(is_isomorphic(s, t))
# Вывод:
True
```

Оценить оптимальность решения по времени и памяти и прикрепить текст кода.

```
def is_isomorphic(s: str, t: str) -> bool:
```

```
    if len(s) != len(t):
```

```
        return False
```

```
    def equal(sym: str):
```

```
        d = {}
```

```
        lst = []
```

```
        for i, s in enumerate(sym):
```

```
            if s not in d:
```

```
                d[s] = i
```

```
                lst.append(d[s])
```

```
        return lst
```

```
    return equal(s) == equal(t)
```

```
s = 'paper'
```

```
t = 'title'
```

```
print(is_isomorphic(s, t))
```

Общая временная сложность: $O(n)$, где n = длина строк, Память: $2 * O(n) = O(n)$

Задание 2: Натуральная последовательность

Реализовать функцию (или тело функции), которая находит единственное отсутствующее число из последовательности натуральных чисел $1, 2, \dots, n$.

```
# Пример:
```

```
nums = [1, 2, 3, 4, 5, 6, 8, 9, 10, 11]
```

```
print(missing_number(nums))
```

```
# Вывод:
```

```
7
```

Оценить оптимальность решения по времени и памяти и прикрепить текст кода.

```
def missing_number(nums: list) -> int:
```

```
    for i in range(1, len(nums) + 2):
```

```
        if i not in nums:
```

return i

Квадратичное время $O(n^2)$ - неоптимально для этой задачи

Лучшее решение:

```
def missing_number(nums: list) -> int:

    n = len(nums) + 1

    return n * (n + 1) // 2 - sum(nums)
```

Время $O(n)$ - оптимально (нужно посмотреть все элементы)

Память $O(1)$ - оптимально (только несколько переменных)

Задание 3: Факторизация

Реализовать функцию (или тело функции), которая при введении натурального числа n разбивает его на простые множители (представить его в виде простых чисел).

```
# Пример:
n = 56
print(prime_factors(n))
# Вывод:
[2, 2, 2, 7]
```

Оценить оптимальность решения по времени и памяти и прикрепить текст кода.

```
def prime_factors(n: int) -> list:

    factors = []

    # обрабатываем делитель 2 отдельно

    while n % 2 == 0:

        factors.append(2)

        n //= 2

    # теперь проверяем только нечётные делители

    d = 3

    while d * d <= n:

        while n % d == 0:

            factors.append(d)

            n //= d

        d += 2 # только нечётные

    if n > 1:
```

```
    factors.append(n)
```

```
    return factors
```

Общая временная сложность: $O(\sqrt{n})$, Память $O(\log n)$ - храним только результат

Блок 3: SQL

Задание 1: Абитуриенты

Есть таблица examination с двумя полями: id (id абитуриента), scores (кол-во набранных баллов дополнительного вступительного испытания от 0 до 100).

Требуется реализовать запрос, который создаёт колонку с позицией абитуриента в общем рейтинге.

В зависимости от того какой рейтинг мы хотим увидеть (с пропуском ранга или без) будет реализована оконная функция **DENSE_RANK()** или **DENSE()**, можно также использовать

```
ROW_NUMBER() DENSE_RANK()  
SELECT  
    id,  
    scores,  
    DENSE_RANK() OVER (ORDER BY scores DESC) AS position  
FROM table  
ORDER BY --сортировка  
LIMIT --ограничение количества выводимых строк  
OFFSET;
```

Задание 2: FULL JOIN

Представьте две таблицы: первая содержит 30 строк, а вторая — 20 строк. Мы выполняем операцию FULL JOIN между ними.

Какой диапазон возможного количества строк может быть в результирующей таблице, если учесть, что ключи для соединения могут быть как полностью совпадающими, так и абсолютно уникальными?

Ответ дать в краткой форме, например: минимально 10 и максимально 3000 строк

ключи совпадающие: минимально 30 и максимально 50

ключи уникальные: у обеих таблиц будет 50 строк

Задание 3: Покупки

```
create table account  
(  
    id integer, -- ID счета  
    client_id integer, -- ID клиента  
    open_dt date, -- дата открытия счета  
    close_dt date -- дата закрытия счета  
)
```

```
create table transaction
```

```
(
  id integer, -- ID транзакции
  account_id integer, -- ID счета
  transaction_date date, -- дата транзакции
  amount numeric(10,2), -- сумма транзакции
  type varchar(3) -- тип транзакции
)
```

Вывести ID клиентов, которые за последний месяц по всем своим счетам совершили покупок меньше, чем на 5000 рублей.

Без использования подзапросов и оконных функций.

```
WITH purchase AS (
  SELECT t.account_id AS client,
         sum(t.amount)
  FROM transaction t
  JOIN account a ON t.account_id = a.client_id
  WHERE t.TYPE = 'PUR'
  GROUP BY t.account_id
  HAVING sum(t.amount) < 50000
)
SELECT client
FROM purchase
ORDER BY client;
```

Блок 4: Статистика и АБ-тесты

Задание 1: Воодушевленное руководство

Вы – аналитик компании Самокат (сервис по доставке продуктов на дом).

Команда решила протестировать гениальную идею замены транспортного средства доставщиков и провели АБ эксперимент в небольшом городке РФ. Результаты превзошли все ожидания: время доставки значительно снизилось в несколько раз! Руководству не терпится применить изменения по всей стране.

Делаем? Выберите все верные утверждения:

- ☐ Делаем. Только применяем изменения в таком же масштабе (количество транспортных средств с изменением), как в городе, где проводили тест.
- ☒ **Тест нерепрезентативен, поэтому результаты применять по всей стране нельзя.**
- ☐ Тест репрезентативен относительно своей генеральной совокупности: таких же небольших городов, можем применять только в подобных городах.